

OpsFlow 部署注意事项

生成环境部署注意事项 — 对项目外部文件的修改清单
2026-06-03

Generated: 2026-06-03 | Source: backend/opsflow/doc/

OpsFlow 对项目有多处侵入式修改（跨 [opsflow/] 目录），部署或迁移时必须逐一处理。 最后更新于：2026-06-01

1. 后端关键修改

1.1 application/settings.py — INSTALLED_APPS

文件：[backend/application/settings.py:70-84]
在 [INSTALLED_APPS] 中添加了以下 OpsFlow 相关项：

App	说明	必需？
`opsflow`	OpsFlow 主应用	是
`pipeline.component_framework`	bamboo-pipeline 组件框架	是
`pipeline.eri`	执行运行时接口	是
`pipeline.contrib.rollback`	执行回滚支持	是
`pipeline.contrib.engine_admin`	pipeline 管理后台	否

****注意：**** [pipeline.contrib.node_timeout] 不可直接注册到 INSTALLED_APPS， 详见下方 § 3.1 特殊说明。

1.2 application/urls.py — API 路由

文件: [backend/application/urls.py:96]

```
path("api/opsflow/", include("opsflow.urls")),
```

所有 OpsFlow API 统一挂载在 [/api/opsflow/] 前缀下, 由 [opsflow/urls.py] 定义具体路由。

1.3 application/routing.py — WebSocket

文件: [backend/application/routing.py]

```
from opsflow.consumers import FlowMonitorConsumer
# ...
re_path(r'^ws/opsflow/execution/(?P<execution_id>\d+)/$', FlowMonitorConsumer.as_asgi()),
```

添加 WebSocket 端点用于实时推送节点执行状态。前端通过此通道接收节点状态变更通知。

1.4 application/asgi.py — ASGI 配置

文件: [backend/application/asgi.py]

ASGI 应用配置了 channels 的 [ProtocolTypeRouter], 同时支持 HTTP 和 WebSocket 协议:

```
application = ProtocolTypeRouter({
    "http": http_application,
    'websocket': AllowedHostsOriginValidator(
        AuthMiddlewareStack(URLRouter(websocket_urlpatterns))
    ),
})
```

****必须使用 Daphne 或 Uvicorn 作为 ASGI 服务器**, 不可使用 gunicorn (gunicorn 不支持 ASGI)。**

1.5 application/__init__.py — Celery 导入

文件: [backend/application/__init__.py]

```
from .celery import app as celery_app
```

****必须保留**, 否则 Django web 进程中 [@shared_task.delay()] 无法获取 Celery broker 配置,**

会回退到默认 AMQP

1.6 application/celery.py — Celery 配置

文件: [backend/application/celery.py]

关键行为:

- [app.set_default_timeout]
- [DJANGO_ALLOW_ASYNC_UNSAFE]
- [@shared_task.retry_delay]

1.7 conf/env.py — 环境配置

文件: [backend/conf/env.py]

```
# OpsFlow — dev 模式自动启动定时调度器 (APScheduler 后台线程)
# 生产环境应运行 python manage.py start_opsflow_scheduler 作为独立进程
OPSFLOW_SCHEDULER_AUTOSTART = True
```

- 开发环境：设为 [True]，Django 启动时自动启动 APScheduler
- **生产环境**：移除或设为 [False]，改用独立进程 [python manage.py start_opsflow_scheduler]

2. 依赖项

2.1 Python 依赖 (requirements.txt)

包	版本	用途
`bamboo-pipeline`	3.29.9	流程引擎内核
`apscheduler`	3.11.2	定时调度
`django-apscheduler`	0.7.0	APScheduler Django 存储后端
`pytz`	2026.1	时区验证

注意事项：

- [bamboo-pipeline] 版本 5.1，属正常依赖
- [pytz] 仅用于

2.2 前端依赖 (package.json)

包	版本	用途
`@antv/x6`	2.19.1	流程画布渲染
`@antv/x6-plugin-clipboard`	2.1.5	剪贴板复制粘贴
`@antv/x6-plugin-dnd`	^2.1.1	拖拽放置
`@antv/x6-plugin-history`	^2.2.4	撤销/重做
`@antv/x6-plugin-keyboard`	2.2.3	键盘快捷键
`@antv/x6-plugin-minimap`	2.0.7	小视窗
`@antv/x6-plugin-selection`	^2.2.2	选区
`@antv/x6-plugin-snapline`	^2.1.7	对齐线
`@antv/x6-plugin-stencil`	^2.1.5	模板侧边栏

3. 特殊注意事项

3.1 pipeline.contrib.node_timeout

****不可****直接注册到 [INSTALLED_APPS]。

原因：[apps.py] 的 [ready()] 方法调用 [hasattr(settings, "redis_inst")], 但 Django 的

[LazySettings] 仅暴

****正确用法****：只在代码中按需 import 后直接调用函数：

```
from pipeline.contrib.node_timeout import apply_node_timeout_configs
apply_node_timeout_configs(tree, timeout_configs)
```

已用 [try/except ImportError] 包裹，导入失败时不影响流程执行。

3.2 Celery Worker 队列

OpsFlow pipeline 使用两个专用队列：

队列	用途
`er_execute`	节点执行任务（bamboo-engine 调度）
`er_schedule`	定时/轮询任务

启动 worker 时必须同时监听这两个队列：

```
celery -A application worker -Q er_execute,er_schedule -l info -P gevent -c 10
```

3.3 APScheduler 调度器

两种启动方式：

方式	设置/命令	适用环境
自启动	`OPSFLOW_SCHEDULER_AUTOSTART = True`	开发
独立进程	`python manage.py start_opsflow_scheduler`	生产

独立进程使用 Redis 锁（[lock:opsflow_scheduler]）防重复启动，锁 TTL 为 60 秒。

3.4 Redis 密码

当前 Redis 服务器无密码。如果后续启用密码，需要修改以下三处：

```
# CACHES OPTIONS — 添加 PASSWORD
"CACHES": {"default": {"OPTIONS": {"PASSWORD": "your_password"}}}

# Celery broker URL
CELERY_BROKER_URL = 'redis://:password@127.0.0.1:6379/0'
CELERY_RESULT_BACKEND = 'redis://:password@127.0.0.1:6379/1'
```

3.5 Redis 版本要求 — BZPOPMIN

[channels_redis.RedisChannelLayer] 内部使用 [BZPOPMIN] 命令（Redis 5.0+ 引入）。

如果 Redis 版本低于

3.6 前端的动态路由

OpsFlow 的前端路由**不**在 [web/src/router/] 中硬编码。

所有页面通过 django

```
后台菜单管理 → 添加菜单 → 选择组件路径
```

3.7 pipeline 表与 opsflow 表分离

- BambooDjangoRuntime 的 Process/Node 记录存储在 MySQL [pipeline_*] 表中
- OpsFlow 的 [FlowExecution]/[OpsLog] 存储在 [opsflow_*] 表中

- 两者通过 [execution_id] 关联，但不共享外键约束

4. 需要额外执行的命令

新部署时必须依次执行：

```
# 1. 数据库迁移
python manage.py migrate opsflow

# 2. 注册菜单项（创建 OpsFlow 所有页面菜单）
python manage.py add_opsflow_menu

# 3. 启动 Celery Worker（监听两个队列）
celery -A application worker -Q er_execute,er_schedule -l info -P gevent -c 10

# 4. 启动 APScheduler 调度器（生产环境用独立进程）
python manage.py start_opsflow_scheduler
```

5. 前端页面目录清单

以下为 OpsFlow 的 6 个前端页面模块，每个模块在后台菜单管理中有对应条目：

页面目录	菜单路径	组件名
`apps/opsflow`	/opsflow	opsflow
`apps/opsflow-template`	/ops/templates	opsflowTemplate
`apps/opsflow-execution`	/ops/executions	opsflowExecution
`apps/opsflow-log`	/ops/logs	opsflowLog
`apps/opsflow-knowledge`	/ops/knowledge	opsflowKnowledge
`apps/opsflow-dashboard`	/ops/dashboard	opsflowDashboard
`apps/opsflow-approval`	/ops/approval	opsflowApproval
`apps/opsflow-webhook`	/ops/webhook	opsflowWebhook
`apps/opsflow-project`	/ops/project	opsflowProject
`apps/opsflow-stats`	/ops/stats	opsflowStats

6. 部署清单

- [] [INSTALLED_APPS] 已包含 [opsflow] 及 pipeline 相关组件
- [] [urls.py] 已配置 [path("api/opsflow/", include("opsflow.urls"))]
- [] [application/routing.py] 已配置 WebSocket 路由
- [] ASGI 服务器已配置（Daphne/Uvicorn）
- [] [python manage.py migrate opsflow] 已执行
- [] [python manage.py add_opsflow_menu] 已执行
- [] Celery worker 已启动并监听 [er_execute] + [er_schedule] 队列
- [] APScheduler 已启动（自启动或独立进程）
- [] Redis 版本 >= 5.0
- [] 前端构建无报错（[npm run build]）
- [] 首次流程测试通过（参见 [注意事项.md] §5）

7. 代码结构变更记录

以下列出 OpsFlow 后端代码的历次重大结构重组，方便追溯：

日期	变更	说明
2026-06-01	<code>`signals.py`</code> → <code>`signals/`</code> 包	单文件拆分为 6 模块（handlers/state/trace/notify/helpers）
2026-06-01	<code>`views/mixins/`</code> 提取	创建 9 个 Mixin 类，template/execution 大 ViewSet 拆分
2026-06-01	<code>`views/dashboard_views/`</code> 包	dashboard_views.py 拆分为 stats/trends/analytics 3 模块
2026-06-01	<code>`core/bamboo_validator.py`</code>	从 bamboo_builder.py 提取 validate_bamboo_compatibility
2026-06-03	<code>`models.py`</code> → <code>`models/`</code> 包	单 models.py 拆分为 11 模块包
2026-06-03	<code>`mixins/`</code> 扩展至 11 个	新增 template_collect, template_webhook 2 个 Mixin
2026-06-03	<code>`views/base.py`</code>	新增 ProjectFilteredViewSet 项目隔离基类
2026-06-03	<code>`mock_view/`</code> 目录	新增 8 组 CMDB Mock 数据端点
2026-06-03	<code>`core/apigw/`</code> 目录	新增外部 API 网关（API Token 认证）
2026-06-03	<code>`signals/timeout.py`</code>	新增超时信号处理器